

PROJETO FINAL · MACHINE LEARNING CLÁSSICO

PC Gamer Price Predictor

Estimativa do preço, em reais, de um computador gamer a partir dos seus 8 componentes, comparando Regressão Linear e KNN

Lorenzo Sartori · João Gustavo · Lucas Rodrigues Vigarani

Prof. Dr. Rodrigo Ramos Silva

Centro Universitário UniSATC
Criciúma/SC · 2026

CONTEXTO

Quanto custa montar um PC gamer?

Montar um PC exige combinar dezenas de peças compatíveis entre si, com preços que mudam de loja para loja.

Antes de pesquisar item por item, o comprador quer uma pergunta respondida: **quanto vai custar essa configuração?**

Nosso objetivo é responder isso com um modelo supervisionado, treinado com preços reais do mercado brasileiro.

8

categorias de componentes

72

peças no catálogo, com preço real

R\$ 2,6 mil →

32,8 mil

faixa de preço das builds no treino

A base teórica: regressão hedônica

Na economia, a regressão hedônica decompõe o preço de um produto no valor de cada uma das suas características. Computadores são o exemplo clássico dessa abordagem.

GRILICHES (1961, 1971)

A origem do método

Propõe usar regressão para relacionar o preço de versões de um produto às suas características.

**BERNDT; GRILICHES;
RAPPAPORT (1995)**

Preços de PCs nos EUA

Journal of Econometrics. Índices hedônicos para PCs apontaram queda média de 28% ao ano nos preços ajustados pela qualidade.

BAR / SCIELO

Desktops no Brasil

Regressão linear hedônica para medir o peso de cada atributo no preço de desktops vendidos no mercado brasileiro.

→ A nossa Regressão Linear com One-Hot é **uma regressão hedônica**: cada coeficiente β representa o valor em R\$ de uma peça.

Machine Learning na predição de preços de hardware

Trabalho	Modelos comparados	Resultado reportado
Shaik et al. (2022)	Árvore, Linear Múltipla, KNN, Random Forest	Linear 80,65% · KNN 80,45%; melhor modelo acima de 88%
Tian (2024)	Linear, Random Forest, XGBoost	XGBoost melhor, em 1.320 laptops
IJSAT (2025)	Linear, Ridge, SVM, KNN, RF, XGBoost e outros	Random Forest melhor
Raykhel; Ventura	KNN com pesos por atributo	Preços de laptops em leilões do eBay

Em dados reais de laptops, **ensembles de árvores costumam vencer** e a Linear Múltipla fica perto de R^2 0,8. Guardem esse número: voltamos a ele na análise crítica.

Os projetos mais próximos do nosso

GITHUB · OSKARBELZA

ComputerPricePrediction

Web scraping de desktops pós-leasing de uma loja polonesa, Regressão Linear com One-Hot e interface para escolher as especificações.

Diferença: PCs usados, atributos genéricos (família da CPU, estado de conservação) e sem compatibilidade.

GITHUB · DACHNHOTK100

pc-recommendation-engine

Web scraping de 5 lojas do Vietnã. Prevê o preço de laptops e PCs montados com LightGBM, CatBoost e Random Forest, e segmenta o mercado com K-Means.

Diferença: PCs prontos de loja, descritos por atributos resumidos (série da CPU, faixa da GPU).

ESTE TRABALHO

PC Gamer Price Predictor

Preço da build nova completa, em R\$, a partir das 8 peças exatas, com socket e DDR compatíveis. Linear × KNN e interface Streamlit.

Contribuição: com preço aditivo, a Linear supera modelos mais complexos.

Nossos resultados × projetos de referência

Projeto	Dados	Modelo	R ²	MAPE
OskarBelza	Desktops usados, Polônia	Regressão Linear	não publicados	
Dachnhotk100	741 PCs montados reais, Vietnã (teste 80/20)	LightGBM (melhor)	0,66	32,9%
CatBoost	0,65	33,1%		
Random Forest	0,59	37,4%		
Este trabalho	3.000 builds sintéticas, preços reais do Brasil (teste 15%)	Regressão Linear	0,991	4,0%
KNN (K = 11)	0,865	12,8%		
Baseline (média)	0,000	56,5%		

Ficamos **bem acima**: MAPE cerca de 8× menor. A diferença vem dos dados, não do modelo: nosso preço é a soma exata das → peças, enquanto em PCs reais há peças e descontos que os atributos não capturam. Comparamos só R² e MAPE porque não dependem da moeda.

DEFINIÇÃO TÉCNICA

Regressão supervisionada: 8 peças → preço em R\$



Entrada: 8 variáveis categóricas

Saída: preço contínuo, em reais

Métricas: MAE (principal), RMSE, R^2 e MAPE

Modelos: Regressão Linear Múltipla e KNN Regressor (scikit-learn)

Critério de escolha: menor MAE na validação

3.000 builds geradas a partir de um catálogo real

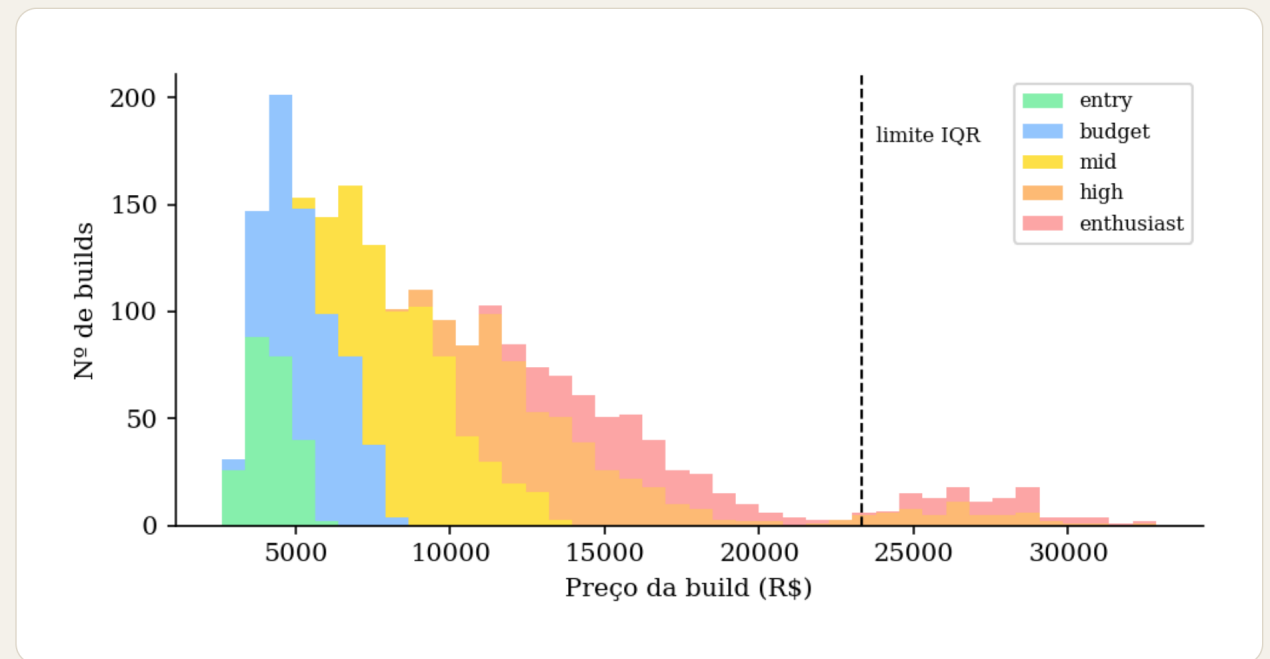
Preços reais de hardwarebarato.com, KaBuM e Pichau (ago/2026)

Builds compatíveis: socket e tipo de DDR são respeitados

5 faixas sorteadas: entry, budget, mid, high e enthusiast

Preço = soma das peças × variação de mercado uniforme entre -8% e +8%

EDA só no treino: 0 nulos e 114 outliers pelo IQR (acima de R\$ 23,3 mil), mantidos por serem builds enthusiast reais



METODOLOGIA

Pipeline sem vazamento de dados

01

Geração

3.000 builds, semente fixa 42

02

Split antes de tudo

70/15/15 estratificado por
faixa: 2.100 / 450 / 450

03

EDA só no treino

Estatísticas, distribuição
e IQR

04

Pipeline + ColumnTransformer

One-Hot ajustado somente
no treino

05

Otimização

GridSearchCV com CV de 5
dobras no treino

06

Seleção

Menor MAE na validação

07

Teste sagrado

Usado uma única vez,
no final

Vazamento de dados
afetou **294 artigos em 17
áreas** e é causa central de
resultados otimistas.

Kapoor; Narayanan (2023)

Uma codificação para cada modelo

REGRESSÃO LINEAR

One-Hot com drop='first'

Categorias ordenadas por preço: a peça mais barata de cada categoria vira a referência, com $\beta = 0$.

$72 - 8 = 64$ coeficientes + intercepto

Cada β passa a ser o **custo extra da peça em R\$** em relação à mais barata.

KNN

One-Hot completo

72 colunas binárias, uma por peça. Não precisa de escalonamento: todas as colunas já valem 0 ou 1.

Remover uma coluna distorceria a distância entre builds.

Duas builds que diferem em **m peças** ficam à distância $\sqrt{2m}$.

MODELO 1

Regressão Linear: o preço como soma das peças

$$\text{preço} = \beta_0 + \beta(\text{CPU}) + \beta(\text{placa}) + \beta(\text{RAM}) + \beta(\text{GPU}) + \dots + \beta(\text{gabinete})$$

Placa de vídeo	Custo extra real	β aprendido
RX 6600	R\$ 350	R\$ 330
RTX 4060	R\$ 1.450	R\$ 1.433
RTX 5070	R\$ 3.413	R\$ 3.338
RTX 5080	R\$ 8.216	R\$ 8.149
RTX 5090	R\$ 17.150	R\$ 16.920

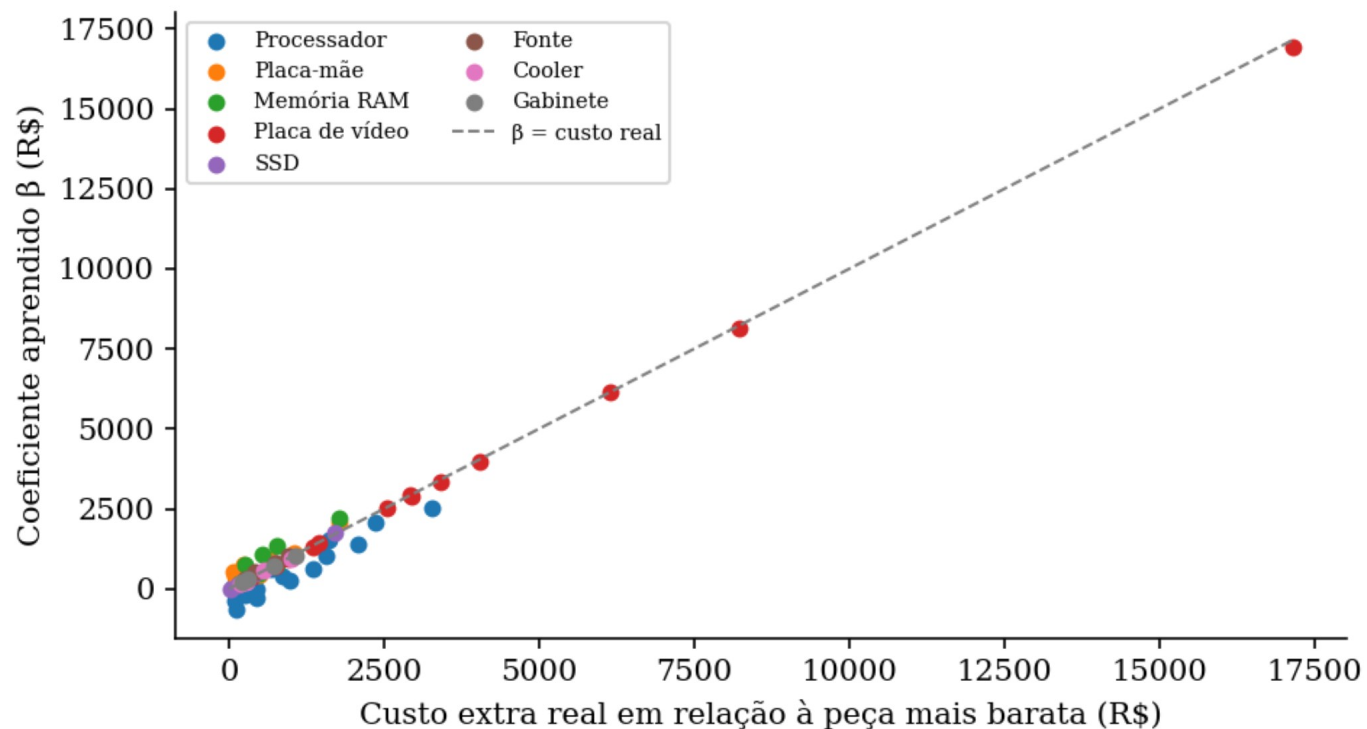
O modelo nunca viu a tabela de preços, só os totais das builds.

Mesmo assim, recuperou o preço de cada placa de vídeo com erro de poucos reais.

$\beta_0 \approx \text{R\$ } 2.451$: o valor da build de referência.

MODELO 1

Coeficientes aprendidos × custo real das peças



Cada ponto é uma peça do catálogo.

Quanto mais perto da diagonal, mais o β bate com o custo real.

Os pontos fora da diagonal são da plataforma CPU, placa-mãe e RAM, que analisamos adiante.

MODELO 2

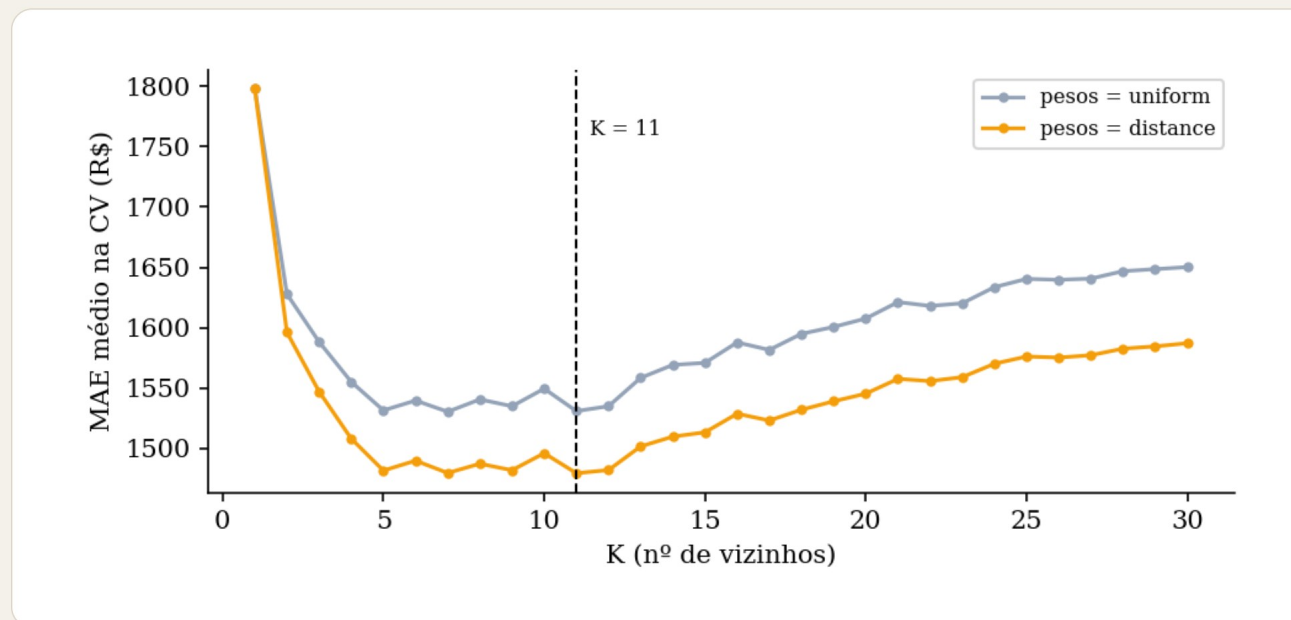
KNN: o preço dos vizinhos mais parecidos

Distância euclidiana sobre as 72
colunas binárias:

Peças diferentes (m)	Distância $\sqrt{(2m)}$
1	1,41
2	2,00
3	2,45

Previsão = média dos K vizinhos, ponderada
pelo inverso da distância.

GridSearchCV: K de 1 a 30 × pesos uniformes
ou por distância, CV de 5 dobras → **K = 11**,
pesos por distância



Resultados

Modelo	MAE CV	MAE validação	MAE teste	RMSE teste	R² teste	MAPE teste
Baseline (média)	—	R\$ 4.430	—	—	—	—
Regressão Linear	R\$ 405	R\$ 431	R\$ 392	R\$ 508	0,991	4,0%
KNN (K = 11)	R\$ 1.480	R\$ 1.427	R\$ 1.342	R\$ 2.021	0,865	12,8%

R\$ 392

de erro médio no teste, para um PC médio de R\$ 10 mil

3,3×

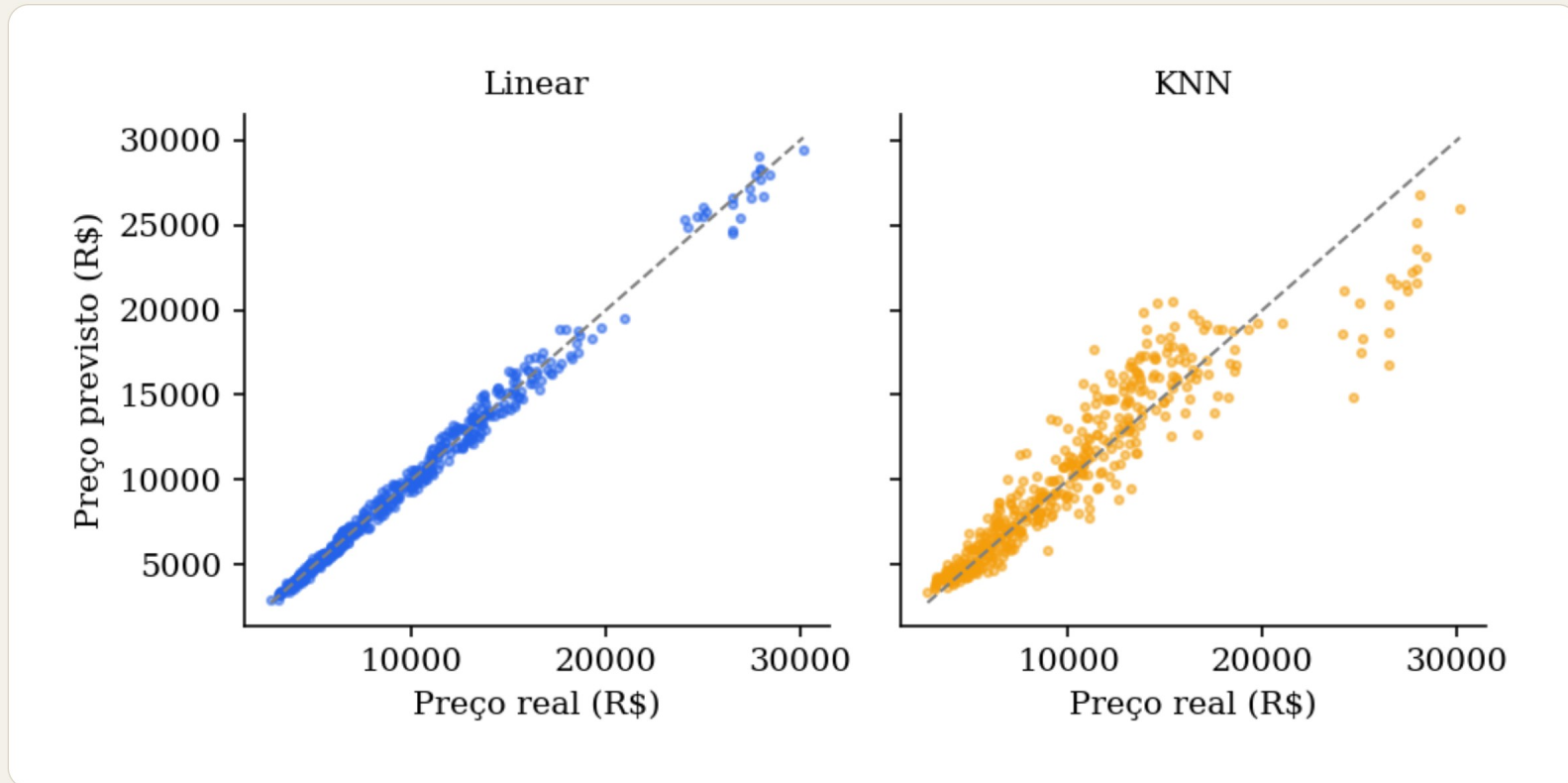
menos erro que o KNN na validação (R\$ 431 × R\$ 1.427)

1 vez

que o teste foi usado, depois da escolha pela validação

AVALIAÇÃO

Valor real × valor previsto no teste



Linear: pontos colados na diagonal em toda a faixa de preço.

KNN: espalha mais e subestima as builds caras, onde há menos vizinhos parecidos.

Por que a Linear venceu?

O preço é aditivo

O preço de um PC é a soma das peças, exatamente a forma que a Linear assume.

O MAPE de 4,0% bate no piso: uma variação uniforme de $\pm 8\%$ tem erro médio de 4%, e esse ruído nenhum modelo remove.

O KNN conta, não pesa

A distância mede quantas peças mudam, não quanto elas custam.

Trocar o gabinete e trocar a placa de vídeo dão a mesma distância, $\sqrt{2}$, embora a diferença de preço seja enorme.

Cuidado com o 0,991

Na literatura, com dados reais, a Linear fica perto de R^2 0,8.

O nosso R^2 alto vem de dados sintéticos aditivos por construção. Em dados reais, espera-se menos.

Um achado: a plataforma CPU + placa-mãe + RAM

Peça (plataforma AM5)	Custo extra real	β aprendido
Ryzen 5 7600	R\$ 452	-R\$ 286
DDR5 16GB 5600MHz	R\$ 260	R\$ 768
MSI MAG B650 Tomahawk	R\$ 645	R\$ 917

A compatibilidade obriga as três peças a aparecerem juntas: processador AM5 só com placa AM5 e memória DDR5.

O modelo não consegue separar o custo de cada uma e redistribui entre elas. **A soma continua certa**, e a previsão do total também.

Conclusão: o β de GPU, SSD, fonte, cooler e gabinete é interpretável sozinho; o do trio, não.

O que o modelo ainda não resolve

Limitações

Dados sintéticos, aditivos por construção

Catálogo fixo de 72 peças: uma peça nova não pode ser prevista

Preços de um único mês (ago/2026)

A variação de $\pm 8\%$ simplifica promoções e diferenças entre lojas

Trabalhos futuros

Coletar builds reais anunciadas, com preço de venda

Testar Ridge para estabilizar os coeficientes da plataforma

Atualizar o catálogo periodicamente a partir do hardwarebarato.com

Comparar com Random Forest em dados reais, como na literatura

DEMONSTRAÇÃO

A interface em Streamlit

Preditor

Monte um PC compatível, com socket e DDR filtrados, e veja a estimativa dos dois modelos.

Resultados

Métricas, real $\times$ previsto, resíduos e EDA do treino.

Teoria

Prova, com a build escolhida, que $\beta_0 + \Sigma\beta$ e a média dos K vizinhos batem com `model.predict()`.

```
python -m streamlit run app_streamlit.py
```

Código, notebook Colab e relatório: github.com/Loren1z9o/Price-Predictory-Hardware-

REPOSITÓRIO

Tudo no GitHub, reproduível

`PC_Gamer_Price_Predictor.ipynb`

Pipeline completo, célula por célula; abre direto no Colab

`pc_gamer_ml.py`

Catálogo, geração dos dados, split, treino, avaliação e baseline

`app_streamlit.py` · `tab_teorias.py`

Interface: abas Preditor, Resultados e Teoria

`relatorio/`

Relatório ABNT, figuras e o build.js que monta o .docx

`README.md`

Resultados, metodologia e como executar



[github.com/Loren1z9o/
Price-Predictory-Hardware-](https://github.com/Loren1z9o/Price-Predictory-Hardware-)

Semente fixa 42 e treino em cerca de
10 s: quem rodar obtém os **mesmos**
números desta apresentação.

CONCLUSÃO

O que aprendemos

- 1 Com um pipeline sem vazamento, a **Regressão Linear** estimou o preço com MAE de R\$ 392 e MAPE de 4%, no limite imposto pelo ruído dos dados.
- 2 A escolha da **codificação** define o que o modelo consegue expressar: β em reais para a Linear, distância $\sqrt{(2m)}$ para o KNN.
- 3 O modelo certo depende da **estrutura do problema**: preço aditivo favorece a Linear, como previa a regressão hedônica.

REFERÊNCIAS

Referências

[AUTORES]. A five-year hedonic price breakdown for desktop personal computer attributes in Brazil. *BAR – Brazilian Administration Review*, [v., n., ano].

BERNDT, E. R.; GRILICHES, Z. Price indexes for microcomputers: an exploratory study. In: FOSS, M. F.; MANSER, M. E.; YOUNG, A. H. (ed.). *Price Measurements and Their Uses*. Chicago: University of Chicago Press, 1993.

BERNDT, E. R.; GRILICHES, Z.; RAPPAPORT, N. J. Econometric estimates of price indexes for personal computers in the 1990's. *Journal of Econometrics*, v. 68, n. 1, p. 243-268, 1995.

GRILICHES, Z. (ed.). *Price Indexes and Quality Change*. Cambridge: Harvard University Press, 1971.

KAPOOR, S.; NARAYANAN, A. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, v. 4, n. 9, 100804, 2023.

OSKARBELZA. *ComputerPricePrediction*. GitHub, 2025. Disponível em: <https://github.com/OskarBelza/ComputerPricePrediction>. Acesso em: 30 set. 2026.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825-2830, 2011.

PHẠM, N. Q. B. *pc-recommendation-engine*. GitHub, 2026. Disponível em: <https://github.com/Dachnhotk100/pc-recommendation-engine>. Acesso em: 30 set. 2026.

SHAIK, M. A. et al. Laptop price prediction using machine learning algorithms. [Evento/periódico], 2022.

TIAN, P. Research on laptop price predictive model based on linear regression, random forest and XGBoost. *Highlights in Science, Engineering and Technology*, v. 85, p. 265-271, 2024.